

Méthodes de Révision de l'architecture Logicielle

Une architecture peut avoir l'air propre sur un diagramme et pourtant être mauvaise dès qu'on lui demande de changer.

C'est exactement à ça que servent les méthodes d'évaluation : faire parler l'architecture avant qu'elle coûte cher.

Les méthodes d'évaluation sont surtout pilotées par des scénarios ou change-cases, proposés et priorisés par les parties prenantes, afin d'identifier tôt les risques, coûts, compromis et zones de complexité.

L'architecture est importante parce qu'elle matérialise les décisions de conception les plus précoces, influence les qualités du système, et sert de support de communication entre les parties prenantes.

Une bonne architecture aide aussi à gérer le changement, ce qui est central puisque beaucoup d'effort logiciel arrive après la mise en production.

Des vocabulaires importants à comprendre:

- Un **scénario** est une situation plausible décrivant un usage ou un changement futur.
- Un **use-case** décrit un comportement attendu du système aujourd'hui.
- Un **change-case** décrit un changement plausible à venir.



Dans les méthodes présentées, les parties prenantes définissent ces cas, les priorisent souvent par vote, puis l'équipe analyse les risques, coûts, compromis et implications architecturales.

Vue d'ensemble en une phrase par méthode

- **SAAM** : "Si le système change, ça casse où ?"
- **ATAM** : "Quels compromis entre performance, sécurité, modifiabilité, etc. ?"
- **CBAM** : "Quel choix architectural vaut vraiment son coût ?"
- **ALMA** : "Combien coûteront les changements futurs et quels sont les points rigides ?"
- **FAAM** : "Une famille de systèmes peut-elle bien s'étendre et bien s'interconnecter ?"

SAAM — Software Architecture Analysis Method

À quoi sert SAAM?

L'objectif initial de SAAM était d'évaluer la modifiabilité d'un système logiciel, bien que certains l'aient étendu à l'examen d'une architecture logicielle selon divers attributs de qualité, notamment la fiabilité, la portabilité, l'extensibilité et les performances.

SAAM est la méthode historique orientée surtout **modifiabilité**. Elle sert à voir si l'architecture supporte bien les changements, à repérer les zones de complexité, et à comparer plusieurs architectures candidates.

Cette méthode fonctionne surtout de manière qualitative.

Scénario d'attribut de qualité logicielle

SAAM est une méthode d'analyse par scénarios, et peut s'avérer efficace pour examiner une architecture logicielle.

Un scénario décrit l'interaction entre une source, telle qu'une partie prenante, et un système logiciel. Il représente une utilisation ou une qualité attendue du système et peut consister en une séquence d'étapes détaillant son utilisation ou sa modification.

Les scénarios permettent de tester les attributs de qualité logicielle, ce qui constitue l'un des objectifs des scénarios d'attributs de qualité logicielle.

Un scénario d'attribut de qualité logicielle comprend les éléments suivants :

- **Source du stimulus** : La source est une entité, telle qu'une partie prenante ou un autre système logiciel, qui génère un stimulus particulier.
- **Stimulus** : Le stimulus est une condition qui exige une réponse du système logiciel.
- **Artefact** : L'artefact est le logiciel stimulé. Il peut s'agir d'une partie du système logiciel, du système logiciel entier ou d'un ensemble de plusieurs systèmes.
- **Environnement** : L'environnement est l'ensemble des conditions dans lesquelles le stimulus se produit. Par exemple, une configuration particulière du logiciel ou des valeurs spécifiques dans les données peuvent être nécessaires à l'existence du stimulus.
- **Réponse** : La réponse est l'activité qui se produit lorsque le stimulus atteint l'artefact.
- **Mesure de la réponse** : Lorsqu'une réponse se produit, elle doit être mesurable. Cela permet de tester la réponse afin de s'assurer que le système logiciel répond aux exigences.

Les étapes

1. Développer les scénarios

On brainstorme avec les parties prenantes pour lister usages actuels et changements futurs plausibles.

2. Décrire l'architecture

On montre les composants, leurs relations, et le comportement général du système.

3. Classer et prioriser les scénarios

On sépare les scénarios en :

- **directs** : l'architecture les supporte déjà
- **indirects** : il faut modifier l'architecture

Puis on les priorise.

4. Évaluer les scénarios indirects

Pour chaque changement, on demande : quels composants faut-il modifier ? ajouter ? combien d'effort ?

5. Évaluer l'interaction entre scénarios

Si plusieurs scénarios touchent les mêmes composants, ces composants deviennent des zones sensibles ou trop chargées.

6. Faire l'évaluation globale

On pondère les scénarios selon leur importance métier et on conclut : architecture acceptable ou pas, meilleure ou moins bonne qu'une autre.

Exemple

→ Les scénarios qu'on peut imaginer avec la plateforme e-commerce :

- ajouter Apple Pay
- gérer la TVA par pays
- ajouter des promotions "flash sale"
- connecter un nouveau transporteur

→ Supposons deux architectures :

- cas A : monolithe classique
- cas B : architecture modulaire avec services séparés pour paiement, promotions et livraison

→ L'analyse qu'on peut faire avec la méthode SAAM :

- "Ajouter Apple Pay" est indirect dans cas A car il touche checkout, paiement, logs, anti-fraude.
- Dans cas B, il touche surtout le service paiement.
- "Nouveau transporteur" touche aussi moins d'éléments dans cas B.
- Si plusieurs changements frappent toujours le module checkout dans cas A, alors checkout devient une zone de forte interaction.

Conclusion

SAAM ne dit pas seulement "ça marche" ou "ça marche pas". Il montre surtout où le changement va coûter cher.

SAAM est très bon pour commencer simplement et faire réfléchir une équipe sur la **modifiabilité**. En revanche, il est moins fort dès qu'on veut analyser finement plusieurs qualités en même temps. Avec SAAM, il n'a pas de métrique très claire et qu'il dépend beaucoup de l'expérience des participants.

ATAM — Architecture Tradeoff Analysis Method

À quoi sert ATAM ?

ATAM reprend l'idée des scénarios, mais va plus loin que SAAM. Il ne regarde pas seulement la modifiabilité. Il analyse aussi d'autres qualités, comme la performance, la disponibilité, la sécurité, l'extensibilité, et surtout les **compromis entre elles**.

Les rôles au sein de la méthode ATAM sont les suivants:

- Team Leader (Coordination et création des équipes d'évaluation)
- Evaluation Leader (conduire la revue)
- Secrétaire des scénarios (prise des notes)
- Rédacteur des comptes rendus (saisir les notes au format électronique)
- Questionneur (soulever les problèmes et de poser des questions)
- Décideurs du projet (apporter des modifications au logiciel si nécessaire et affecter ou approuver les ressources nécessaires)
- Parties prenantes (toute personne ayant un intérêt direct dans l'architecture logicielle et le système)

Les étapes

ATAM est souvent présenté en 4 phases et 9 étapes.

Phase 1 — Présentation

1. Présenter la méthode
2. Présenter les **business drivers**
3. Présenter l'architecture

Phase 2 — Analyse

4. Identifier les approches architecturales
5. Construire le **utility tree**
6. Analyser les approches architecturales

Phase 3 — Test

7. Brainstormer et prioriser les scénarios
8. Réanalyser l'architecture avec les scénarios prioritaires

Phase 4 — Restitution

9. Présenter les résultats

Les notions clés

- Un **sensitivity point** est un endroit de l'architecture très sensible à une qualité.
 - Exemple : la base de données influence fortement la performance.
- Un **trade-off point** est un point sensible pour plusieurs qualités en même temps.
 - Exemple : renforcer les contrôles de sécurité peut dégrader un peu la performance.
- Un **risk** est un point qui peut empêcher le système d'atteindre ses objectifs qualité.

Exemple

- Business drivers (objectifs métier) :
 - site très rapide
 - paiement très sécurisé
 - disponibilité pendant les soldes
 - ajout rapide de nouveaux moyens de paiement
- On construit un utility tree avec des scénarios comme :
 - "Pendant le Black Friday, 10 000 utilisateurs simultanés, temps de réponse < 2 secondes"
 - "Une tentative de fraude doit être bloquée sans bloquer les paiements légitimes"
 - "Ajouter un nouveau moyen de paiement en moins de 5 jours"
- Approches architecturales possibles:
 - cache distribué
 - file de messages pour certaines tâches
 - service paiement isolé
 - WAF / anti-fraude plus strict

Ce qu'ATAM révèle selon ces cas :

- la base de données peut être un **sensitivity point** pour la performance
- le moteur anti-fraude peut devenir un **trade-off point** entre sécurité et latence
- un checkout trop centralisé peut être un **risk** pour la modifiabilité et la disponibilité

Quality attribute Utility Tree

ATAM une méthode d'évaluation par scénarios. Il sert à comprendre comment l'architecture satisfait les qualités attendues, mais surtout comment ces qualités se **combinent** et se **mettent en tension**.

Les attributs de qualité qui composent l'"utility" du système sont d'abord identifiés, puis déclinés en scénarios, annotés avec des **stimuli** et des **réponses**, puis **priorisés**.

Ensuite, les scénarios les plus prioritaires servent à identifier les **risques**, **non-risques**, **sensitivity points** et **trade-off points**.

Le système: App E-Commerce

→ Business drivers du système. Le métier veut :

- un site rapide
- un site disponible pendant les pics de charge
- un paiement sécurisé
- une évolution rapide des fonctionnalités
- une expérience correcte sur mobile et web

ATAM prend en entrée l'architecture, les objectifs métier et les points de vue des parties prenantes, et qu'il utilise différents types de scénarios : use case, growth et exploratory scenarios.

Exemple : Quality attribute Utility Tree pour une plateforme E-commerce

- Utility
 - └─ ## Performance
 - └─ Temps de réponse catalogue
 - └─ Temps de réponse paiement
 - └─ ## Availability
 - └─ Tolérance aux pannes de service
 - └─ Continuité pendant pic de charge
 - └─ ## Security
 - └─ Protection du paiement
 - └─ Détection / limitation des abus
 - └─ ## Modifiability
 - └─ Ajout d'un nouveau moyen de paiement
 - └─ Ajout d'une nouvelle règle promotionnelle
 - └─ ## Usability
 - └─ Parcours de commande fluide
 - └─ Dégradation maîtrisée sur mobile

ATAM exploitable dans un tableau avec des Mésures de Réponse

Attribut	Sous-attribut	Scénario	Type	Réponse attendue	Mesure de réponse	Priorité
Performance	Temps de réponse catalogue	5 000 utilisateurs consultent simultanément les fiches produit pendant une campagne promotionnelle	Use case	le système continue à servir les pages produit sans saturation visible.	95 % des requêtes en moins de 2 secondes.	Haute
Performance	Temps de réponse paiement	800 utilisateurs valident leur panier dans la même minute.	Use case	le service de paiement reste réactif et ne crée pas d'attente excessive.	temps moyen de validation < 3 secondes, taux d'échec technique < 1 %	Très haute
Disponibilité	Tolérance aux pannes	le service "recommandations" tombe pendant les heures de pointe	Exploratoire	le site reste utilisable, sans bloquer la navigation ni le panier.	le catalogue, le panier et le checkout restent disponibles ; seule la recommandation est dégradée.	Haute
Disponibilité	Continuité sous charge	le trafic est multiplié par 10 pendant 20 minutes au lancement d'une vente flash.	Exploratoire	le système continue à fonctionner avec dégradation contrôlée.	pas d'arrêt complet, maintien des fonctions critiques, file d'attente acceptable.	Très haute
Securité	Paiement sécurisé	plusieurs tentatives de paiement avec cartes compromises sont détectées sur une courte	Use case / failure	le système bloque ou isole les transactions suspectes sans bloquer les transactions légitimes.	transactions suspectes bloquées en temps réel ; faux positifs < seuil défini par le métier.	Très haute
Securité	Protection des comptes	un attaquant envoie des milliers de tentatives de connexion sur plusieurs comptes.	Exploratoire	le système limite les tentatives et protège les comptes.	verrouillage progressif, rate limiting, traçabilité des événements, pas d'impact majeur sur les autres utilisateurs.	Haute
Modifiabilité	Ajout moyen de paiement (Apple Pay)	le métier demande l'ajout d'un nouveau moyen de paiement avant la prochaine campagne marketing.	Growth	le nouveau moyen de paiement est intégré sans réécriture du checkout complet.	modification localisée ; délai de mise en œuvre < 5 jours ; tests de non-régression limités.	Très haute
Modifiabilité	Ajout règle promo	le marketing demande une promotion "2 achetés = 1 offert" sur une catégorie précise.	Growth	la règle peut être ajoutée sans modifier plusieurs modules critiques à la fois.	Impact limité à moteur de promotion + configuration ; pas de changement profond du panier ni du paiement.	Haute
Utilisabilité	Parcours commande mobile	un utilisateur passe commande depuis un réseau mobile lent.	Use case	le parcours reste compréhensible, lisible et sans blocage.	checkout en moins de 5 étapes, messages d'erreur clairs, chargement progressif acceptable.	Moyenne/Haute
Utilisabilité	Dégradation UX	les images secondaires ou recommandations chargent lentement.	Exploratoire	l'utilisateur peut quand même rechercher, ajouter au panier et payer.	fonctions essentielles toujours accessibles	Moyenne

Une fois l'arbre fait, vous ne vous arrêtez pas là. Dans l'ATAM, les scénarios à plus forte priorité sont pris comme base pour analyser les approches architecturales. C'est à ce moment qu'on cherche :

- les **risques**
- les **non-risques**
- les **sensitivity points**
- les **trade-off points**

Exemple d'analyse rapide à partir de cet arbre

→ Points Sensibles :

- Le **checkout** et le **service paiement** sont des sensitivity points, car une petite décision de conception à cet endroit influence fortement la performance, la sécurité et la disponibilité. Un **sensitivity point** est un ensemble de composants critiques pour l'atteinte d'un attribut de qualité.

→ Trade-off point

- Le moteur **anti-fraude** peut devenir un trade-off point :
 - plus de contrôle = meilleure sécurité
 - mais aussi plus de latence = moins bonne performance
 - et potentiellement plus de friction = moins bonne usabilityC'est exactement la logique ATAM : C'est de révéler les interdépendances et les trade-offs entre qualités.

→ Risk

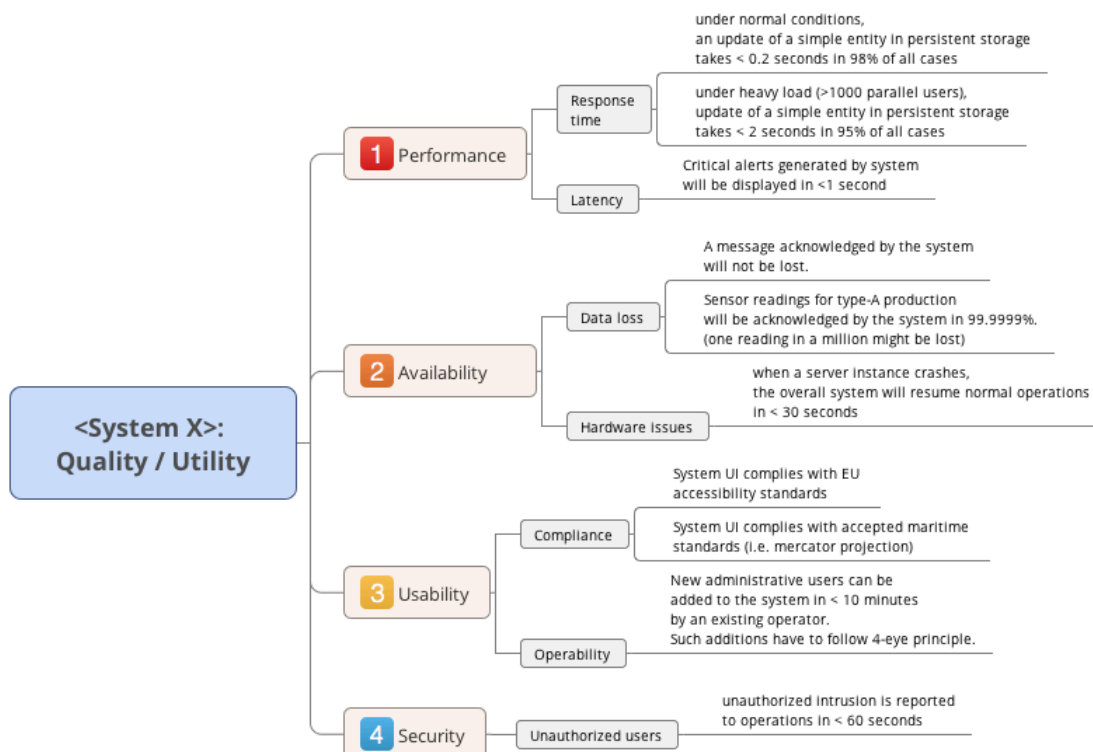
- Si le checkout est fortement couplé au moyen de paiement actuel, alors le scénario Modif-1 "ajouter Apple Pay" devient risqué.
 - Conséquence : faible modifiabilité, délai long, risque de régression élevé.

→ Non-risk

- Si le catalogue est déjà servi via cache/CDN et que les scénarios Perf-1 sont couverts par des tests de charge réalistes, alors la performance sur consultation produit peut être considérée comme un non-risk provisoire.

Donc, le **utility tree** ne sert pas juste à lister des qualités. Il force l'équipe à traduire des attentes floues comme 'le système doit être rapide' en scénarios testables et priorisés. Ensuite, ATAM s'en sert pour repérer où l'architecture est solide, où elle est fragile, et où une qualité nuit à une autre."

Autre exemple de "Utility Tree"



Conclusion

ATAM sert quand on ne veut plus juste savoir "où ça change", mais "quel compromis on accepte entre plusieurs qualités".

ATAM est la bonne méthode quand le débat n'est plus seulement technique mais stratégique : performance contre sécurité, rapidité de livraison contre propreté de conception, etc. Les supports indiquent aussi que c'est la méthode la plus forte pour mettre en évidence les trade-offs.

CBAM — Cost Benefit Analysis Method

À quoi sert CBAM?

CBAM commence là où ATAM s'arrête. Il ajoute la dimension **économique** : coûts, bénéfices, délais, incertitude. En clair, il aide à choisir non pas l'architecture "la plus belle", mais celle qui a le meilleur rapport valeur/coût.

Les étapes

- Choisir les scénarios qui préoccupent le plus les parties prenantes, et les stratégies architecturales associées
- Évaluer les bénéfices liés aux qualités
- Quantifier les bénéfices des stratégies
- Quantifier les coûts et impacts planning
- Calculer la désirabilité
- Prendre la décision

Exemple

- Tu dois choisir entre trois investissements :
 - stratégie A : cache distribué pour accélérer le site
 - stratégie B : moteur anti-fraude avancé
 - stratégie C : nouveau moteur de recherche produit
- Tu demandes :
 - quel bénéfice sur performance, sécurité, conversion, satisfaction client ?
 - quel coût ?
 - quel délai ?
 - quelle incertitude ?

Exemple pédagogique

- A coûte 30k, améliore fortement performance et conversion
- B coûte 20k, améliore fortement sécurité mais peu la conversion
- C coûte 50k, améliore conversion mais avec forte incertitude

CBAM pousse l'équipe à dire : "Dans notre contexte business, A est peut-être la meilleure désirabilité maintenant."

Conclusion

CBAM transforme une discussion d'architectes en décision d'investissement. CBAM est également utile quand l'entreprise demande : "D'accord pour améliorer l'architecture, mais combien ça coûte et qu'est-ce qu'on y gagne ?" Les supports soulignent justement qu'il fournit une base rationnelle pour décider et qu'il relie architecture et ROI.

ALMA — Architecture-Level Modifiability Analysis

À quoi sert ALMA?

ALMA reste focalisée sur la **modifiabilité**, mais avec une approche plus structurée que SAAM pour trois objectifs possibles :

- prédire les coûts de modifications futures
- identifier les rigidités du système
- comparer plusieurs architectures

Les étapes

1. Définir l'objectif de l'analyse
2. Décrire l'architecture
3. Eliciter les change-scenarios
4. Évaluer l'impact de ces scénarios
5. Interpréter les résultats

→ Le point fort d'ALMA

L'**impact analysis** : on regarde quels composants sont touchés et quels effets en cascade apparaissent. C'est très concret pour une équipe de développement.

Exemple

L'entreprise sait qu'elle devra probablement :

- gérer plusieurs entrepôts

- vendre en abonnement
- appliquer des règles de prix par pays
- connecter un ERP externe

→ Objectif ALMA : prévoir l'effort de maintenance sur les 12 prochains mois

→ L'analyse qu'on peut faire en tant qu'architect :

- si "prix par pays" touche catalogue, checkout, facturation, reporting, alors le changement est large
- si "nouvel ERP" crée des effets en cascade dans stock, commandes, logistique, alors l'architecture est rigide à cet endroit
- si l'architecture modulaire limite ces effets, la maintenance future coûtera moins cher

Conclusion

ALMA est très bon quand on veut parler du futur travail de maintenance, pas seulement de la conception d'aujourd'hui. ALMA est pertinent pour des discussions très terrain : "où sera l'effort demain ?", "quel design nous coûtera moins cher à faire évoluer ?". A noter aussi que la précision des prédictions reste limitée, donc il faut le présenter comme un outil d'aide à la décision, pas comme une vérité mathématique.

FAAM — Family-Architecture Assessment Method

À quoi sert FAAM

FAAM ne vise pas un seul système, mais une **famille de systèmes**. Il s'intéresse surtout à deux qualités :

- Interopérabilité
- Extensibilité

Les étapes

1. Définir l'objectif de l'évaluation
2. Préparer les exigences qualité de la famille
3. Préparer l'architecture
4. Revoir et affiner les artefacts
5. Vérifier la conformité de l'architecture
6. Restituer les résultats et propositions

Exemple

Une entreprise possède :

- un site e-commerce web
 - une application mobile
 - un portail partenaires
 - une borne magasin
 - une API logistique
- Elle veut une **architecture de famille** commune.

→ Questions que FAAM posent :

- peut-on ajouter facilement une nouvelle application partenaire ?
- les systèmes échangent-ils bien leurs données ?
- un nouveau module livraison peut-il être réutilisé partout ?
- la famille supporte-t-elle l'extension vers un modèle franchise ?

→ FAAM va vérifier si la famille de systèmes respecte bien ces besoins d'interopérabilité et d'extensibilité.

Conclusion

FAAM est utile quand le problème n'est plus "mon application", mais "mon écosystème de produits". FAAM est spécialisé. Il est moins pertinent pour un produit isolé, mais très utile si l'organisation construit plusieurs systèmes proches ou variantes d'un même socle.

Comparer les méthodes

- **SAAM** : je veux tester la facilité de changement.
- **ATAM** : je veux comprendre les compromis entre plusieurs qualités.
- **CBAM** : je veux relier architecture et décision économique.
- **ALMA** : je veux estimer l'impact et les coûts des changements futurs.
- **FAAM** : je veux évaluer une famille de systèmes, surtout pour l'interopérabilité et l'extensibilité.

Comparaison rapide des méthodes

	Qualités	Métriques & outils	Support du processus	Forces	Faiblesses	Applicabilité
SAAM	Modifiabilité	Classification des scénarios (directs vs. indirects)	Raisonnable	- Identification des zones à fort potentiel de complexité - Ouvert à toute description architecturale	- Ce n'est pas une métrique de qualité claire- Non soutenu par des techniques pour réaliser les étapes	Général
ATAM	Modifiabilité	Points de sensibilité, points de compromis, pris en charge par l'outil ATA	Bon	Applicable aux propriétés statiques et dynamiques	Nécessite des connaissances techniques détaillées	Général
CBAM	Coûts & bénéfices, implications sur le planning	Temps et coût	Raisonnable	Fournit des mesures métier en intégrant l'incertitude	Nécessite des connaissances commerciales détaillées	Général
ALMA	Modifiabilité	Estimation d'impact, modifiabilité, modèle de prédiction	Raisonnable	Critère d'arrêt pour la génération de cas de changement	Se concentre sur les propriétés statiques, peu d'études de cas	Général
FAAM	Interopérabilité & extensibilité	Divers tableaux et diagrammes spécialisés	Très bon	Donne aux équipes les moyens d'évaluer leur propre travail	Se concentre sur les propriétés statiques, pas réellement prouvé	Familles de systèmes

- Support du processus : désigne à quel point la méthode aide concrètement à mener l'évaluation de bout en bout:
 - **Very Good / Très bon** : la méthode guide bien les étapes, les rôles, les livrables, les analyses à faire.
 - **Good / Bon** : le processus est assez bien structuré, avec une démarche exploitable.
 - **Reasonable / Raisonnable** : il existe un certain support, mais il reste incomplet ou peu détaillé.